Performance Engineering of a Spark Pipeline on the NYC Yellow Taxi Trip Records

Rodrigo Loução Nº73984 & Guilherme Ferreira Nº78037
ComPar 2026/27 - Project Assignment 1

1 Introduction

We analyse the NYC Taxi & Limousine Commission yellow-cab trip records [2] for 2024 (41.2 M trips, 12 monthly Parquet files, 515 MB) with a PySpark 4.0.1 pipeline [1]. We started by cleaning the data and enforcing a schema (*M1*), then we performed a temporal demand analysis (*M2*) and zone-level statistics with window functions (*M3*). Additionally, we chose two optional operations: origin-destination flow analysis (*O2*) and demand-forecasting features (*O3*). Each operation independently consumes the cleaned trips produced by *M1* (Table 1). This workload is interesting from a performance standpoint because every operation reads the same 41 M trips, *O2* groups a high-cardinality key (49 k routes) and *O3* needs a self-join, so it exposes repeated scans, shuffles and join strategies. We profiled it on an Apple M4 laptop, on a Spark standalone cluster in Docker and on three nodes of the university's DI cluster, and implemented every optimisation as a switch in the code (Section 4).

2 Pipeline

Table 1: Pipeline operations and the Spark features each one exercises.

Op	What it computes	Main Spark features / cost drivers
M1	Schema enforcement, 7 validity criteria, per-year removal counts	Scan + filter, single-pass aggregate
M2	Trips, mean fare/duration/distance per (borough, weekday, hour)	Broadcast join, group-by, partitioned write
M3	Per zone-month stats; rank, rolling 3-month share, 1.5σ flag	Aggregate then 4 window specs
O2	Monthly route counts, outbound share, top-20, Q1/Q3 reversals	High-cardinality shuffle, pivot, 2 joins
O3	Hourly zone demand, historical mean/variance, 4-week lag	Self-join, join-back of bucket statistics

Architectural decisions. We used the *DataFrame API* for all operations: helper functions (time features, zone joins) are shared by *M2*–*O3*, Spark SQL would compile to the same Catalyst plan, and RDDs would lose column pruning and filter pushdown. Each monthly file is read separately and mapped to a canonical schema, because column names and types change across years, and the 12 scans are unioned. Each operation is a function that returns a lazy DataFrame, and only its write is profiled; the baseline deliberately has no caching. *M3* and *O2* aggregate before their windows and joins (3 060 and 352 k rows instead of 39 M). The pipeline is a notebook driven by papermill (command-line parameters), the session time zone is UTC (identical results locally and on the cluster), and the Lab 3 profiler was extended with per-stage durations, a task-skew ratio and JSON export. Figure 1 shows the plan of *M2*.

Data-quality decisions. *M1* removes 2 210 549 of the 41 169 720 records of 2024 (5.4%; the criteria overlap): `fare_amount` ≤ 0 (748 284), `trip_distance` ≤ 0 (776 305), `passenger_count` $= 0$ or > 6 (401 638), dropoff $\leq$ pickup (13 510), location outside 1–263 (425 660) and, as an added criterion, pickups outside the month of their source file (420, with dates from 2002 to 2026, which would bias the per-weekday means of *O3* by up to 7%). No required field is null. The 4.09 M trips (9.9%) without `passenger_count` are kept: they also lack `RatecodeID` and have `payment_type` 0, but are otherwise valid, the brief's rule does not cover missing values, and no later operation uses the field. The *M3* flag compares a month with the *previous* 3 months (with the current month included, a 3-value window can never exceed 1.15σ), using month indices (`rangeBetween`) so that missing months count as 0 trips. In *O2*, a reversal means that the Mar−Jan and Sep−Jul share changes have opposite signs, reported for the top-20 routes (by total trips in 2024). In *O3*, means and variances include zero-trip hours; averaging only the observed hours overstates the median bucket 2.5×.

```
HashAggregate (final)          <- averages
+- Exchange hashpartitioning   <- shuffle, 2.2 MB
   +- HashAggregate (partial)  <- per-task pre-sum
      +- BroadcastHashJoin     <- + borough
         :- Union of 12 x Filter (M1) <- Scan parquet [7 cols]
         +- BroadcastExchange <- Scan csv (zones, 265 rows)
```

Figure 1: Simplified physical plan of *M2* (read bottom-up), from `explain(mode="formatted")`. Spark reads 7 of the 19 columns, broadcasts the zone table and pre-aggregates before the only shuffle.

3 Profiling and Baseline

Experimental setup. The laptop is an Apple M4 (10 cores, 16 GB RAM). *Local*: `local[*]`, 8 GB driver, Java 21. *Cluster* (Docker, used in Sections 3–4): Spark standalone in Docker with 1 master and 3 workers × 2 cores (1536 MB executors, 2 GB driver, Java 17), all sharing the laptop's 10 CPUs and 7.75 GB. *DI cluster*: up to 3 identical nodes of the university cluster (32 cores, 125 GB RAM each) running the same Spark 4.0.1 in standalone mode from a Python environment in the shared home folder, with one

worker of 16 cores and 48 GB per node (40 GB executors) and the master and driver on the first node; the input is read from the shared home folder, and the outputs of the operations are computed but not written (Spark's `noop` sink). Non-default settings: `spark.sql.session.timeZone=UTC` and, on the cluster, `spark.scheduler.minRegisteredResourcesRatio=1.0` with `spark.cores.max` set to the alive cores, so jobs only start once every executor has registered. Each configuration runs 3 times after a discarded warm-up run (median reported), with only one environment running at a time. Metrics come from the Spark REST API via our profiler [3].

Table 2: Baseline per operation: wall time (s, median of 3 runs), tasks, shuffle write (MB) and skew (max/median task time of the worst stage), run r1. Every operation reads the 515 MB input once, except *O3* (2 575 MB).

	Local (10 cores)				Cluster (3×2 cores)			
Op	Time	Tasks	Shuffle	Skew	Time	Tasks	Shuffle	Skew
M1	4.0	123	0.0	35×	9.9	75	0.0	5.0×
M2	7.7	123	2.2	72×	13.9	75	2.2	2.9×
M3	6.6	126	2.2	78×	16.6	78	2.2	2.4×
O2	9.7	195	90.0	130×	15.1	122	86.1	2.7×
O3	36.0	672	119.3	380×	72.5	416	116.8	4.3×
Total	63.3 (range 62.8–72.5)				121.1 (range 111.0–153.5)			

Bottlenecks.

- *Repeated scans*: The same 515 MB are scanned 9 times per run (4.6 GB and 371 M rows, for 39 M valid trips): *M1*, *M2*, *M3* and *O2* once each and *O3* five times, because intermediate DataFrames are lazy and recomputed by every consumer. The scan stages are 91% (local) and 94% (cluster) of all the task time.
- *O3's performance*: It accounts for 56% (local) and 57% (cluster) of the runtime, with 5 concurrent scans that compete for the cores and a double exchange before the self-join. On the cluster its stage durations form a staircase (25 to 81 s) while the median task takes only 0.7–2.3 s, so the stage duration overstates the work.
- *Empty read tasks*: Each of the 12 per-file scans is split into one piece per core, but the files only have 3–4 Parquet row groups of ≈16.5 MB, so locally most read tasks are empty (median 10–34 ms, slowest 1.2–3.8 s; skew 35–380×, only 2–5× on the cluster).
- *O2 high cardinality*: Reading is only 66–83% of *O2*'s task time; the rest is grouping 49 k routes, windows, pivot and joins, with 86–90 MB shuffled.
- *Sort-merge joins on tiny tables*: 2 in *O2* and 3 in *O3*, against small tables whose size Spark cannot estimate before running.

Other usual bottlenecks were measured and are not a problem here: waiting for shuffle data takes at most 0.02 s per operation, there is no spill, garbage collection is 2.5–4.7% of the task time and the cores are busy 74–95% of the time. This means that the pipeline is slow because it does too much work, mainly reading.

4 Optimisation Study

Each optimisation is a switch in the code (all switches off is the baseline). Each configuration ran 3 times locally and 3 times on the cluster, next to a control run of the baseline in the same session (local 69.1 s, cluster 95.8 s, median); on the cluster the configurations were interleaved. The speed of the machine changes between runs (up to 17% for operations that a switch does not change), so these unaffected operations are used as an extra check. The outputs of every configuration are identical to the control's. This check also found that the top-20 of *O2* changed between runs because of ties in quiet boroughs, which we fixed by breaking ties with the zone IDs.

4.1 Reading the input once

(a) Bottleneck. The same 515 MB are scanned 9 times per run, and the scan stages are 92% of the task time. **(b) Hypothesis.** Reading the Parquet files once and keeping them in memory removes 8 of the 9 scans, which should reduce the runtime a lot; building the cache will make *M1* slower. **(c) Change.** The 9 columns used by *M1*–*O3* are cached, and the statistics of *M1* fill the cache, so its cost is included in *M1*. **(d) Result.** The total goes from 69.1 s to 33.9 s (2.04× faster) and the task time from 621 s to 300 s. *M1* is slower (3.7 s → 8.6 s), but *M2* (8.3 s → 3.3 s), *M3* (7.5 s → 2.8 s), *O2* (9.8 s → 7.1 s) and *O3* (41.7 s → 12.7 s) are faster. On the cluster the total goes from 95.8 s to 48.3 s (1.98×) and *O3* from 55.6 s to 16.1 s. The hypothesis is confirmed.

4.2 O3: caching the intermediate table

(a) Bottleneck. *O3* is 58% of the runtime: its `hourly` table is used 5 times and computed again from the input each time. **(b) Hypothesis.** Caching the intermediate tables reused inside an operation (`hourly` in *O3*, 5 uses; `routes` in *O2*, 3 uses) avoids computing them again. **(c) Change.** A first version cached both: *O3* became faster (36.7 s → 19.9 s), but *O2* almost two times slower (9.2 s → 17.2 s, from 195 to 1 371 tasks), because Spark already reuses the shuffle of *O2*, and a cached table keeps all the 200 shuffle partitions (`spark.sql.optimizer.canChangeCachedPlanOutputPartitioning` is `false` by default). The final version

only caches `hourly` and sets this option to `true`. **(d) Result.** *O3* goes from 41.7 s to 10.0 s (it reads 578 MB instead of 2 575 MB and runs 269 tasks instead of 672) and the total from 69.1 s to 37.6 s (1.84× faster); *O2* is unchanged (9.6 s). On the cluster the total goes from 95.8 s to 46.1 s (2.08×). Together with Section 4.1, the total is 27.8 s locally (2.49×) and 37.7 s on the cluster (2.54×). The hypothesis is confirmed.

4.3 Empty read tasks

(a) Bottleneck. 120 read tasks per scan locally, but only 3–4 row groups per file (skew up to 226×). **(b) Hypothesis.** Splits of one row group remove the empty tasks and balance the read stage. **(c) Change.** `spark.sql.files.maxPartitionBytes=16m` and `spark.sql.files.minPartitionNum=1`. **(d) Result.** The tasks go from 1 239 to 582 and the skew from 226× to 3.8×, but the total only changes from 69.1 s to 64.2 s (61.9 s → 61.9 s in a first batch), within the variation of the machine. Rejected: the empty tasks only took milliseconds, so the skew was high but did not cost time.

4.4 O3: one shuffle less before the self-join

(a) Bottleneck. `hourly` is shuffled twice: aggregated by (zone, date, weekday, hour) but self-joined by (zone, date, hour). **(b) Hypothesis.** Aggregating by exactly the join keys lets the join reuse that shuffle. **(c) Change.** Aggregate by (zone, date, hour) and derive the weekday from the date afterwards. **(d) Result.** *O3*'s shuffle goes from 119 MB to 101 MB and *O3* from 41.7 s to 36.5 s, but the unaffected operations also became 11% faster, so the real gain is close to zero. Rejected: the removed shuffle is small next to the 5 scans of *O3*.

4.5 Sort-merge joins on tiny tables

(a) Bottleneck. Sort-merge joins against small tables Spark cannot size before running (2 in *O2*, 3 in *O3*); AQE switches some to broadcast only after shuffling the large side. **(b) Hypothesis.** Broadcast hints avoid shuffling the large side of these joins. **(c) Change.** `F.broadcast` on *O2*'s top-20 (120 rows) and trend (49 k rows) and *O3*'s `n_days` (7 rows) and `stats` (44 k rows). **(d) Result.** The shuffle goes from 214 MB to 145 MB (*O2*: 90 MB → 41 MB) and the total from 69.1 s to 55.9 s, but *M1–M3*, which do not use these joins, were also 17% faster, and a first batch showed no gain (61.9 s → 63.8 s). Rejected for time: waiting for shuffle data was already ≈0 s.

All optimisations combined. With all five switches the total is 25.1 s locally (2.76× faster), compared with 27.8 s with only optimisations 1 and 2: once the repeated scans are removed, the shuffles and the number of tasks become a bigger part of what is left, and optimisations 3–5 give a further 10%. On the cluster, all five give 38.7 s (2.48×), the same as 1 and 2 alone (37.7 s), because with only 6 cores the read tasks were not empty in the first place. On the DI cluster with 3 nodes, the optimised pipeline takes 34.8 s compared with 63.9 s for the baseline (1.84× faster; 2.42× on one node).

Table 3: Summary of optimisations (median of 3 runs). "Before" is the control run of the same session. Optimisations 3–5 were not run alone on the cluster, because locally they showed no real gain.

		Local (s)		Cluster (s)		Key metric
Optimisation	Targets	Before	After	Before	After	before → after
1. Reading the input once	M1–O3	69.1	33.9	95.8	48.3	scans 9 → 1
2. Caching O3's `hourly`	O3	69.1	37.6	95.8	46.1	O3 41.7 → 10.0 s
3. Read splits = row groups	scans	69.1	64.2	–	–	skew 226 → 3.8×
4. Aligned self-join keys	O3	69.1	64.5	–	–	O3 shuffle 119 → 101 MB
5. Broadcast hints	O2, O3	69.1	55.9	–	–	shuffle 214 → 145 MB
1 + 2		69.1	27.8	95.8	37.7	2.49× / 2.54×
All combined		69.1	25.1	95.8	38.7	2.76× / 2.48×

5 Analysis

Dataset scale. We ran the optimised pipeline (all five switches) on the Docker cluster with real subsets of the data (Figure 2); we used subsets instead of a random sample because the sample is taken after reading, so the files would still be read completely. Up to one year the time grows sub-linearly: 14.2 s for 1 month (3.0 M trips, 7% of the year) and 38.7 s for one year (41.2 M trips), so the time per million trips falls from 4.8 s to 0.94 s. A straight line through these points crosses zero trips at about 12 s, the cost that does not depend on the input size: starting the jobs and stages, broadcasting the zone table, running the small final stages and writing the outputs. Two years (89.9 M trips, 2.18× one year) take 182.7 s, 4.7× one year, which is much worse than linear: the cache no longer fits in the memory of the executors, so it is written to their local disks (≈2 GB each), and garbage collection grows from 3% to 9% of the task time. Each size was run once, except one year (3 runs).

Cluster size. We computed the speedup $S(n) = T(1)/T(n)$ and the efficiency $E(n) = S(n)/n$ on the full year (Figure 3). On the DI cluster, with 1 to 3 identical nodes (`charmander-3` through `charmander-5`), the baseline goes from 109.7 s to 63.9 s: $S = 1.33$ with 2 nodes ($E = 0.67$) and 1.72 with 3 ($E = 0.57$). The network is not the reason for the gap to the ideal: 97–124 MB of shuffle data cross between the nodes, but waiting for it takes

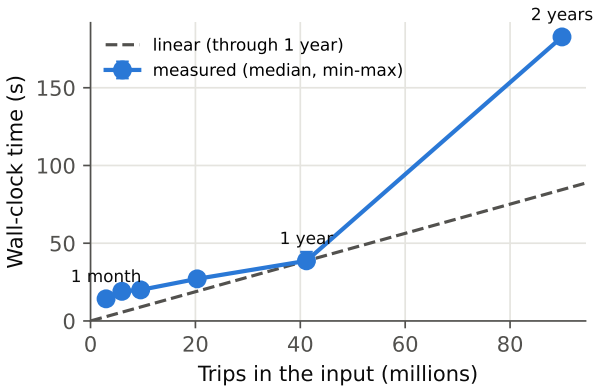


Figure 2: Wall-clock time of the optimised pipeline on the Docker cluster (3 workers) for 1, 2, 3, 6 and 12 months of 2024 and for 2024+2025. Dashed: linear scaling through the one-year point.

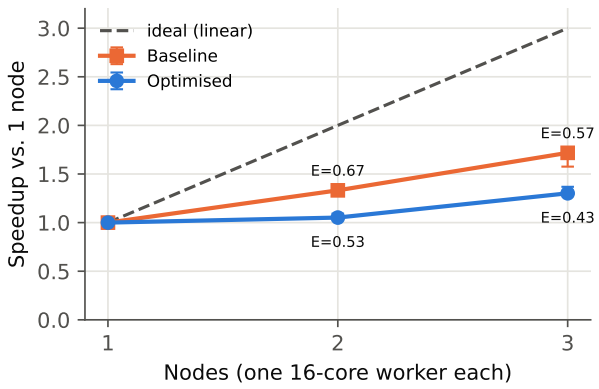


Figure 3: Speedup on the DI cluster (2024) with 1–3 nodes, one 16-core worker each, relative to 1 node (median of 3 runs, min–max bars); E is the efficiency. Dashed: ideal.

at most 0.13 s. Instead, distributing the work costs extra task time (1 380 s with 1 node, 1 713 s with 3), and the cores are busy a smaller share of the time (79% → 56%), because the late stages have few tasks. The optimised pipeline scales less ($S = 1.05$ and 1.30; 45.3 s → 34.8 s): once the repeated scans are removed, most of what is left is fixed or serial work, as Amdahl's law predicts; for example, *M2* and *M3* take ≈5 s each with any number of nodes. One DI node (16 cores) is slower than the 10-core laptop (109.7 s vs. 63–69 s for the baseline), because the same work takes 1 380 s of task time instead of 621 s, with the tasks busy on the CPU 88–93% of the time. This means that reading from the shared home folder is not the bottleneck: the laptop's cores are simply ≈2.2× faster on this workload.

6 Discussion

Remaining bottlenecks. After the optimisations, building the cache in *M1* is the most expensive step (28% of the runtime locally, 35% on the cluster). The cached year takes 1.4 GB in memory, compared with 515 MB of Parquet on disk, which is most of the ≈2.2 GB the 3 executors have for execution and storage; with two years, each executor keeps 0 MB of it in memory and ≈2 GB on disk. *O2* is now 25% of the runtime, because its top-20 and trend branches each read the cached table again (4.2 GB read from memory), so it would have to be restructured to compute its results from a single aggregation.

Limitations. The Docker cluster shares the laptop's 10 cores (4 performance and 6 efficiency) and memory and has no real network, so we used the DI cluster for the scaling study; there we could only use 3 nodes, the runs do not write the outputs, and one year of data leaves little work for 48 cores. The timings varied between sessions (up to 17%; the cluster control between 82 and 146 s), mainly because of the memory pressure of the laptop. Same-session controls, interleaving and unaffected operations reduce this, but with 3 runs (1 for most scaling points) differences below ≈10% cannot be trusted. The optimisation study only uses 2024, and the 2009–2010 files (different schema, no location IDs) were not processed.

With more time. We would run the two-year dataset and more nodes on the DI cluster, where the cache fits in memory, to see whether the optimised pipeline scales better with more work per core. We would also rewrite the input once into a better layout, keeping only the 9 used columns and partitioning the files by year and month, which is cheaper than building the in-memory cache and allows reading subsets directly. Finally, we would restructure *O2* and repeat every experiment more times on a machine without other load.

Extra page

References

[1] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. Spark SQL: Relational data processing in spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015.

[2] NYC Taxi & Limousine Commission. TLC trip record data. `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`. Yellow taxi trip records 2024, Parquet; accessed October 2026.

[3] The Apache Software Foundation. Monitoring and instrumentation. `https://spark.apache.org/docs/4.0.1/monitoring.html`. Spark 4.0.1 documentation.

Acknowledgements

We used the help of AI tools to scaffold the notebook structures, the profiler extensions, to explain Spark concepts (such as window functions, broadcast joins and shuffle), to help run experiments and to help write the report. All design decisions, optimisations, results and the text were reviewed by the authors.

Individual contributions

Member	Work	Share
Rodrigo Loução 73984	M1–M3, O2, optimisations 1–2, report sections 3–4	60%
Guilherme Ferreira 78037	O3, optimisations 3–4, scalability runs, report sections 1–2 & 5–6	40%

Comments and suggestions

Nothing to add.